

Introduction

Kadane's algorithm is one of the first "aha" moments most people have with dynamic programming. Track a running sum, reset it when it goes negative, keep the best value you've seen — done. It solves maximum subarray sum in one clean pass, and because it works so reliably, it becomes the reflexive answer any time "maximum subarray sum" shows up in a problem statement.

That reflex is exactly what breaks here. Add one constraint — the subarray's length has to be a multiple of k — and Kadane's algorithm keeps producing an answer, it just produces the *wrong* one. It has no concept of length at all; it only cares about sum. That gap between "runs without error" and "runs correctly" is what makes this problem worth sitting with. It's a good test of whether you actually understand why Kadane's algorithm works, or whether you've just memorized the five lines of code.

This pattern — prefix sums bucketed by a property of the index rather than scanned as one flat array — shows up again in interviews under different disguises: subarrays with sum divisible by k , subarrays of exactly length k , subarrays where two elements must be a fixed distance apart. Once you see the "lane" idea once, you start recognizing it everywhere.

Problem Overview

You're given an array of integers and an integer k . You need to find the maximum possible sum of a contiguous subarray, but with one restriction: the subarray's *length* must be divisible by k . So valid lengths are k , $2k$, $3k$, and so on — never anything in between.

The array can hold up to 200,000 elements, which rules out any solution that checks subarrays one at a time. You need something close to linear time.

If the array has fewer than k elements, there's no valid subarray at all — there isn't even one slice of length k to pick. Return null or raise, depending on your API's convention.

Example

```
nums = [2, -1, 5, -3, 4]
k = 2
```

Valid lengths: 2 or 4 (the array only has 5 elements, so 2 or 4 are the only multiples of k that fit).

- Length 2 candidates: $[2, -1]=1$, $[-1, 5]=4$, $[5, -3]=2$, $[-3, 4]=1$
- Length 4 candidates: $[2, -1, 5, -3]=3$, $[-1, 5, -3, 4]=5$

Best overall: $[-1, 5, -3, 4]$ with sum 5 .

Notice Kadane's algorithm, run unmodified on this array, would greedily assemble $[2, -1, 5]$ or similar length-3 windows if they scored higher — a length that isn't a multiple of k at all, and therefore not a legal answer.

Intuition

The instinct when you see "subarray sum" is to reach for Kadane. Resist it here — Kadane collapses every subarray into a single running value and forgets what length it corresponds to. Once length itself is part of the constraint, you need a technique that can filter by length cheaply, and rescanning every possible length is where the n^2 brute force comes from.

The move that unlocks this is prefix sums. Build $\text{prefix}[i]$ as the sum of the first i elements, with $\text{prefix}[0] = 0$ representing the empty prefix. Now the sum of any subarray from index i to $j-1$ (inclusive) is just:

```
subarray_sum(i, j) = prefix[j] - prefix[i]
```

That turns "find the best subarray" into "find the best pair of prefix values to subtract." And the length of that subarray is simply $j - i$.

Here's the part that makes the constraint tractable: $j - i$ is divisible by k exactly when i and j share the same remainder mod k . That's just modular arithmetic — if two numbers differ by a multiple of k , they land on the same point going around a clock with k hours on it.

So instead of comparing every prefix index against every other prefix index, you only ever need to compare indices that share a remainder. Split the indices into k separate "lanes," one per remainder value. Within a lane, the problem is now the classic "maximize $\text{prefix}[j] - \text{prefix}[i]$ for $i < j$ " — which you solve by keeping a running minimum as you scan left to right.

It's worth being explicit about a trap here: this is *not* the same setup as Continuous Subarray Sum, where you track the running sum's value mod k . There, the sum's divisibility is what matters, so you bucket by $\text{sum} \bmod k$. Here, it's the *length* that must divide k , so you bucket by $\text{index} \bmod k$ — the running sum itself stays a normal, un-modded total. Mixing these two up is the single most common way people get this problem wrong.

Brute Force Solution

Idea: For every valid length (multiples of k up to n), slide a window of that length across the array and track the best sum found.

Algorithm: 1. For each length L in $\{k, 2k, 3k, \dots\}$ up to n 2. For each starting index, sum the L elements starting there 3. Track the running maximum

Advantages: Trivial to reason about and verify by hand; no modular arithmetic to get wrong.

Disadvantages: Recomputing each window's sum from scratch is $O(n)$ per window and there are $O(n)$ windows per length, with $O(n/k)$ valid lengths — worst case (k small) this is $O(n^2)$. On 200,000 elements that's tens of billions of operations, far past what runs in a reasonable time limit.

Complexity: Time $O(n^2)$ in the worst case, space $O(1)$.

(You can shave this to $O(n^2/k)$ with a sliding sum per length, but it's still quadratic-shaped for small k , so it doesn't change the fundamental problem.)

Optimal Solution

The optimal approach has three pieces, all built on the intuition above:

1. **Prefix array** — $\text{prefix}[0] = 0$, then $\text{prefix}[i] = \text{prefix}[i-1] + \text{nums}[i-1]$.
2. **One minimum tracker per remainder lane** — $\text{lane_min}[r]$ holds the smallest prefix value seen so far among indices where $\text{index} \% k == r$.
3. **A single forward pass** — at each index j , compute its lane $r = j \% k$. If that lane already has a recorded minimum, $\text{prefix}[j] - \text{lane_min}[r]$ is a candidate answer for a subarray ending at j . Then update $\text{lane_min}[r]$ with $\text{prefix}[j]$ if it's smaller than what's stored.

Step	What happens	Why
Seed	$\text{lane_min}[0] = 0$	$\text{prefix}[0]$ belongs to remainder 0 and must be seedable immediately
Scan	Walk j from 1 to n	Each j is a candidate "end" of a subarray
Compare	$\text{prefix}[j] - \text{lane_min}[j \% k]$	Only ever compares against an earlier index in the <i>same</i> lane, guaranteeing the length divides k
Update	$\text{lane_min}[j \% k] = \min(\text{lane_min}[j \% k], \text{prefix}[j])$	Keeps the lane's minimum current for future comparisons

Because each lane only ever needs its running minimum — not every value it has ever seen — the whole thing collapses into one pass with `k` numbers of extra memory.

Python Code

```
def max_subarray_sum_div_by_k(nums: list[int], k: int) -> int | None:
    """Max sum of a contiguous subarray whose length is a multiple of k."""
    n = len(nums)
    if k <= 0 or n < k:
        return None

    lane_min = [None] * k
    lane_min[0] = 0 # prefix[0] = 0, remainder 0

    prefix = 0
    best = float("-inf")

    for j in range(1, n + 1):
        prefix += nums[j - 1]
        lane = j % k

        if lane_min[lane] is not None:
            best = max(best, prefix - lane_min[lane])

        if lane_min[lane] is None or prefix < lane_min[lane]:
            lane_min[lane] = prefix

    return best if best != float("-inf") else None
```

Code Walkthrough

- `lane_min = [None] * k` — one slot per remainder; `None` means "no prefix has landed in this lane yet."
- `lane_min[0] = 0` — seeds the empty-prefix case so a subarray starting at index 0 is comparable from the very first iteration.
- The loop runs `j` from 1 to `n` inclusive, representing `prefix[j]` — the sum of the first `j` elements.
- `prefix += nums[j - 1]` extends the running total by the newest element.
- `lane = j % k` decides which bucket this prefix belongs to.
- The candidate check `prefix - lane_min[lane]` only fires once that lane has a recorded minimum — otherwise there's no earlier index to pair with.

- The update at the end always keeps `lane_min[lane]` as the smallest prefix seen so far in that lane, which is what makes future subtractions maximal.

Dry Run

`nums = [2, -1, 5, -3, 4]`, `k = 2`

j	nums[j-1]	prefix	lane (j%2)	lane_min before	candidate	best	lane_min after
1	2	2	1	None	—	-inf	lane1=2
2	-1	1	0	0	1-0=1	1	lane0=0 (unchanged)
3	5	6	1	2	6-2=4	4	lane1=2 (unchanged)
4	-3	3	0	0	3-0=3	4	lane0=0 (unchanged)
5	4	7	1	2	7-2=5	5	lane1=2 (unchanged)

Final answer: `5`, matching `[-1, 5, -3, 4]` from the earlier example.

Complexity Analysis

Time: $O(n)$ — a single forward pass, constant work per index.

Space: $O(k)$ — one minimum value stored per remainder lane, independent of `n`.

Both are correct because the algorithm never revisits an index and never needs more than the running minimum per lane — it doesn't need to remember every prefix value, just the smallest one in each bucket, since only the smallest can ever produce the maximum difference.

Alternative Solutions

For the *fixed-length* version of this problem (subarray length exactly `k`, not a multiple of `k`), a plain sliding window is the right tool — add the incoming element, drop the outgoing one, no prefix sums needed. It's tempting to reach for that here too, but sliding window assumes a single known length; once the length is "any multiple of `k`," you'd need a separate window per length, which regresses back toward brute force. The remainder-lane trick only earns its keep once the length is unknown ahead of time.

Edge Cases

- **Array shorter than `k`** — no valid subarray exists; return `None` /raise, don't silently return 0.

- **All negative numbers** — you still must return a subarray of valid length; the algorithm naturally finds the "least bad" option since it just maximizes the difference, without ever skipping the requirement.
- **k equals the array length** — only one valid length exists (the whole array), so the answer is simply `sum(nums)`.
- **Best answer is the entire array** — the lane logic handles this without special-casing; it's just the largest possible gap between `prefix[0]` and `prefix[n]` if they share a lane.
- **k = 1** — every length is valid, degenerating into the classic unconstrained maximum subarray sum problem.

Common Mistakes

1. **Applying Kadane's directly** — it ignores length entirely and will happily return an invalid-length subarray.
2. **Bucketing prefix sums by `sum mod k` instead of `index mod k`** — that's the setup for Continuous Subarray Sum, a different problem with a different divisibility target.
3. **Forgetting to seed `lane_min[0] = 0`** — without it, subarrays starting at index 0 in remainder-0 positions get silently skipped.
4. **Off-by-one on the prefix array** — using `nums[j]` instead of `nums[j-1]`, or looping `j` from `0` instead of `1`, throws off every remainder calculation.
5. **Not guarding `n < k`** — the loop will run without error but return a meaningless or misleading result instead of signaling "no valid subarray."

Interview Questions

- How would this change if you needed the length to be *exactly* a multiple of `k`, but with an upper bound?
- What if instead of "length divisible by `k`," the constraint were "length at least `k`"?
- Can you reconstruct the actual subarray (indices), not just the sum?
- How would you handle a streaming input where you can't hold the whole array in memory?

Similar Problems

- **Maximum Subarray (LeetCode 53)** — the unconstrained base case; Kadane's algorithm alone.
- **Continuous Subarray Sum (LeetCode 523)** — bucket by `sum mod k` instead of `index mod k`; easy to conflate with this problem.

- **Subarray Sum Equals K (LeetCode 560)** — prefix sums plus a hashmap of counts, same prefix-difference idea in a different shape.
- **Maximum Sum of Subarray with Length K (fixed-length sliding window)** — the simpler sibling this problem is often mistaken for.

Key Takeaways

A tool working correctly and a tool being the *right* tool are different things — Kadane's algorithm runs fine here, it just answers a question you didn't ask. The fix isn't a new algorithm from scratch; it's reframing the sum constraint as a prefix-difference problem, then noticing that the length constraint becomes a remainder constraint on the indices. Once indices are grouped into lanes by `index % k`, the rest is the same running-minimum trick that solves plain maximum subarray sum, applied `k` times over instead of once.

Watch the Video

For the full walkthrough — including a live trace over three official test cases plus extra edge cases, and a real timing comparison between brute force and the optimal solution — watch the video here: {LINK}

About the Series

This is part of Fun with Learning Technology, a series that breaks down one coding interview problem at a time — not just the solution, but the reasoning that gets you there, the wrong turns that look tempting, and why they fail. New problem every episode, same format: understand the trap first, then the fix.

Call To Action

If this pattern clicked, the video walks through it visually with a live trace and timing comparison — worth a watch on YouTube, and hitting subscribe means you won't miss the next one. For deeper write-ups like this one delivered straight to your inbox, subscribe to the Substack. Questions, corrections, or a case this doesn't cover? Drop a comment — and if this saved you from a debugging rabbit hole, share it with someone prepping for interviews.

Quick Review

Kadane's Trick fails here — why?

Kadane greedily extends/resets any subarray, but it can't enforce a length constraint like 'length % k == 0'.

Core reframe: subarray sum via prefix sums?

$\text{sum}(i..j) = \text{prefix}[j] - \text{prefix}[i-1]$. Fixing $\text{length} \% k$ means $i-1$ and j must land in the same remainder class mod k .

'Same remainder, same lane' — what's the trick?

Group prefix sums by $\text{index} \% k$ into k lanes; track min prefix seen per lane, maximize $\text{prefix}[j] - \text{minPrefix}[\text{lane}]$ as j scans.

Time and space complexity of the prefix-lanes solution?

$O(n)$ time — one pass building prefix sums and updating per-lane minimums; $O(k)$ space for the lane minimums array.

Trickiest edge case in the lane-minimum trick?

Must initialize lane 0 with $\text{prefix}[0]=0$ (empty prefix) before scanning, or you miss subarrays starting at index 0.

This lane-by-remainder trick vs plain prefix-sum + hashmap (subarray sum equals k)?

Both index prefix sums by a key; here the key is $\text{index} \% k$ (fixed k lanes), not the sum value itself.